

The maximum size of an almost-equidistant set in $\mathbb{R}^4$ is twelve

A computer-assisted resolution of a conjecture of
Balko, Pór, Scheucher, Swanepoel and Valtr

August 5, 2026

Background

A finite set of points in $\mathbb{R}^d$ is *almost equidistant* if among any three of its points, some two are at unit distance. Let $f(d)$ denote the largest size of an almost-equidistant set in $\mathbb{R}^d$. Classically $f(2) = 7$ (the Moser spindle, uniqueness due to Talata), and

$$f(3) = 10,$$

proved by **Bernadett Györey** in her 2004 master’s thesis [1] (supervised by István Talata), together with the uniqueness of the extremal ten-point configuration — two unit tetrahedra with two reflected vertices, one copy rotated about the axis through the reflected points. This result is Theorem 2 of the paper of Balko, Pór, Scheucher, Swanepoel and Valtr [2], who reproved it with computer assistance, established

$$12 \leq f(4) \leq 13,$$

and conjectured (their Conjecture 1) that $f(4) = 12$. The lower bound comes from the $(2d+4)$ -point generalization of the ten-point configuration; the upper bound from their enumeration of candidate graphs. The gap has remained open.

Result. $f(4) = 12$. *Conjecture 1 of [2] holds (computer-assisted proof; see the caveats below).*

Reduction

Following [2], the unit-distance graph of a hypothetical thirteen-point almost-equidistant set in $\mathbb{R}^4$ must be an *abstract almost-equidistant graph*: a graph on 13 vertices with no independent set of size 3, no K_6 , and no $K_{1,3,3}$. There are exactly **59** such graphs. We re-enumerated them independently; our counts agree with Table 3 of [2] at every order (7, 14, 37, 97, 316, 934, 2362, 2814, 944, **59**, 4, 1, 1, 0 for $n = 4, \dots, 17$). Thus $f(4) = 13$ if and only if one of these 59 graphs admits a *realization*: an assignment of *13 distinct* points of $\mathbb{R}^4$ placing every edge at distance exactly 1 (non-edges are unconstrained). We certified that **none of the 59 graphs admits such a realization**. Appendix A details the enumeration, Appendix B the certified decision procedure, and Appendix C the search orchestration.

Method (overview)

For each graph, a branch-and-prune search over *rigorous interval arithmetic* (outward-rounded IEEE double precision) explores all possible placements up to congruence:

- a seed clique (K_5 or K_4 , a unit simplex, rigid up to congruence) is placed with exact interval enclosures;

- a vertex with ≥ 4 placed neighbours lies on the intersection of four unit spheres — at most two points, giving a binary branch tree, with enclosures contracted by the Krawczyk operator and surplus neighbours acting as certified pruning constraints;
- a vertex with exactly 3 placed neighbours lies on a circle, parametrized exactly and subdivided adaptively; near-tangent configurations are resolved by a one-dimensional subdivision along the root segment.

A branch is discarded only when an interval certifiably excludes a required value, so eliminating *all* branches proves non-realizability.

A key subtlety — echoing §3.5 (“multiple points”) of [1] — is that these graphs *do* admit degenerate realizations in which some non-adjacent vertices coincide; such configurations have fewer than 13 distinct points and are irrelevant, but they satisfy every edge constraint and would survive naive pruning. Two certified rules (Appendix B.6) discard branches on which coincidence is *forced*.

Validation. The engine kills the ten-vertex graph proved non-realizable in Lemma 12 of [2], and does *not* kill the realizable cross-polytope graph $K_{2,2,2,2}$ — for which it isolates exactly the two mirror-image unit cross-polytopes. During development the engine was maintained in two independent implementations (a Python reference and a C production version, roughly $1000\times$ faster), which produced identical search trees node-for-node on matched instances; the reproduction package ships the C implementation. The C engine’s determinism was additionally verified *across platforms*: on a matched battery of instances — including full searches of both control graphs — Apple clang/libm on ARM64 (M2 Pro, macOS 14.5), MSVC/UCRT on x86-64, and LLVM clang/UCRT on x86-64 (Ryzen 9950X3D, Windows 11) produce bit-identical search trees, node and unresolved-cell counts agreeing exactly both on zero-circle instances (exercising only correctly rounded basic operations) and on circle-bearing instances (additionally exercising the padded library cos/sin enclosures). During development a genuine bug in the circle parametrization (a Gram–Schmidt error) was caught by manual inspection of a single cell; a runtime canary — aborting the computation if a parametrized circle point ever violates its own defining constraint — was added in response, and every result predating the fix was discarded and recomputed.

Execution. Fifty-six of the 59 graphs die quickly (milliseconds to minutes). The three hardest — including the unique graph requiring a two-parameter search — were finished by partitioning the circle parameter into certified slices computed in parallel on 12 CPU cores; one graph exhibited a degenerate tangency channel specific to the chosen elimination order and was instead certified in full by an alternate order, for which all 24 slices covering the whole circle die. In total: **zero surviving branches anywhere**. Independent numerical optimization (Levenberg–Marquardt, 200 restarts per graph) corroborates the verdict: no graph comes close to realizability (best residual 0.078, versus 10^{-31} for a realizable control).

Caveats and reproducibility

The proof is computer-assisted. It assumes IEEE-754 correctly-rounded floating point arithmetic and standard-library cos/sin accurate to within 8 units in the last place (padded accordingly), and the correctness of the verification engine described in Appendix B. A self-contained reproduction package (all source code, the enumerated graphs, and a one-command driver) accompanies this note, so that the computation can be replicated or re-implemented independently — which we would warmly encourage before the result is treated as established.

With $f(2) = 7$, $f(3) = 10$ [1], and now $f(4) = 12$, the first four values of f are settled.

References

- [1] B. Györey, *Diszkrét metrikus terek beágyazásai*, Master’s thesis, Eötvös Loránd University, Budapest, 2004.
- [2] M. Balko, A. Pór, M. Scheucher, K. Swanepoel, P. Valtr, *Almost-equidistant sets*, *Graphs and Combinatorics* **36** (2020), 729–754. arXiv:1706.06375.
- [3] A. Kupavskii, N. Mustafa, K. Swanepoel, *Bounding the size of an almost-equidistant set in Euclidean space*, *Combinatorics, Probability and Computing* **28** (2019), 280–286.

A Enumeration of the candidate graphs

(Implemented in `enumerate_aeq.py`.) A graph G is an abstract almost-equidistant graph in $\mathbb{R}^4$ if (i) $\alpha(G) \leq 2$, i.e. the complement is triangle-free (this is the almost-equidistance condition itself); (ii) G contains no K_6 (at most $d+1 = 5$ points of $\mathbb{R}^4$ are pairwise at unit distance); and (iii) G contains no complete tripartite $K_{1,3,3}$ (Lemma 11 of [2]).

Generation. Graphs are grown one vertex at a time from a single vertex. When a graph on n vertices is extended by a new vertex v , we may assume v has minimum degree in the extended graph: all three defining properties are inherited by induced subgraphs, so every $(n+1)$ -vertex witness arises from deleting a minimum-degree vertex. Moreover $\deg(v) \geq n - 5$, because the non-neighbourhood of any vertex must induce a clique (two non-adjacent non-neighbours would form an independent triple with v), and cliques have size ≤ 5 by (ii). For each admissible neighbourhood the three conditions are checked *incrementally*, i.e. only for configurations involving v : the non-neighbours of v must be pairwise adjacent; no 5-clique of the parent may lie inside $N(v)$ (else a K_6); and a $K_{1,3,3}$ through v is detected by testing, for v as the singleton and for v inside a triple, whether some triple $X \subseteq N(a)$ has at least three common neighbours inside $N(a) \setminus X$ (a bitmask intersection-popcount test).

Isomorphism rejection. Candidates are bucketed by the invariant $\{(\deg u, \# \text{triangles at } u, \text{sorted neighbours of } u)\}$ and compared within buckets by a backtracking isomorphism search with degree pruning.

Verification. Every generated graph is re-checked by independent, non-incremental tests of (i)–(iii). The resulting counts match [2] Table 3 for every n (the sequence quoted in the main text; in particular there are 59 graphs for $n = 13$ and none for $n \geq 17$), and the counts of *minimal* graphs (those in which deleting any edge creates an independent triple; an edge uv is deletable iff $N(u) \cup N(v) \cup \{u, v\}$ is not the whole vertex set) match their Table 2, e.g. 12 minimal graphs on 13 vertices. Although killing the 12 minimal graphs would suffice (every realization of an abstract graph restricts to one of a spanning minimal subgraph), we certified all 59.

B The certified decision procedure

(Implemented in `ckernel.c`, compiled and driven by `cdriver.py`; the exact seed enclosures of §B.2 are constructed in `ival.py`.)

B.1 Interval arithmetic

All geometry is computed in interval arithmetic over IEEE doubles with *outward rounding*: after each basic operation $(+, -, \times, /, \sqrt{})$, which IEEE-754 rounds correctly), the bounds are widened by one ulp via `nextafter`. Library `cos/sin` are additionally padded by 8 ulp and interval extrema at multiples of π are handled explicitly. The invariant throughout: *every quantity’s interval contains the exact real value for every real configuration represented by the current branch*. A constraint is violated — and the branch discarded — only when its interval excludes the required value entirely.

B.2 Normalization and seeds

Every realization can be moved by an isometry so that a chosen clique (K_5 if present, else K_4 ; both exist in all 59 graphs) occupies the canonical unit simplex

$$u_0 = 0, \quad u_1 = e_1, \quad u_2 = (\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}, 0, 0), \quad u_3 = (\tfrac{1}{2}, \tfrac{\sqrt{3}}{6}, \tfrac{\sqrt{6}}{3}, 0), \quad u_4 = (\tfrac{1}{2}, \tfrac{\sqrt{3}}{6}, \tfrac{\sqrt{6}}{12}, \tfrac{\sqrt{10}}{4}),$$

placed as exact interval enclosures. (For a K_4 seed both mirror images are explored by the branching itself.)

B.3 Sphere placements

If v has placed neighbours $p_0, \dots, p_{k-1}$ with $k \geq 4$, subtracting the sphere equations pairwise yields the linear system $2(p_i - p_0) \cdot x = \|p_i\|^2 - \|p_0\|^2$ ($i = 1, 2, 3$), solved by interval Gauss–Jordan elimination with mignitude pivoting; the solution set is a line $x = x_p + t n$, and substituting into $\|x - p_0\|^2 = 1$ gives a quadratic $at^2 + bt + c = 0$. If its discriminant interval is negative the branch dies; if it straddles zero the two roots are returned as one merged enclosure (see B.5); otherwise two root enclosures give a binary branch. Each candidate is contracted by the *Krawczyk operator*

$$K(X) = m - YF(m) + (I - YJ(X))(X - m),$$

where m is the midpoint box, F the four sphere residuals, J their interval Jacobian, and Y a floating-point approximate inverse of the midpoint Jacobian (the best-conditioned 4-subset of neighbours is chosen by maximal $|\det|$). $K(X) \cap X = \emptyset$ certifiably kills the branch; otherwise the intersection is a valid, usually much tighter, enclosure. All remaining placed neighbours act as extra certified constraints. After each placement inside a circle cell, a Gauss–Seidel sweep re-contracts the neighbourhood (constraints propagate backwards as well as forwards).

B.4 Circle placements

If v has exactly three placed neighbours p_0, p_1, p_2 , the solution set of the three unit-sphere equations is a circle: its centre c (the circumcentre of p_0, p_1, p_2 in their affine plane) solves a 2×2 interval system, its radius is $r = \sqrt{1 - R^2}$ with $R = \|c - p_0\|$ (negative $1 - R^2$ kills the branch), and an orthonormal basis u, w of the orthogonal complement of the plane is obtained by interval Gram–Schmidt. Then $x(\theta) = c + r(\cos \theta u + \sin \theta w)$, $\theta \in [0, 2\pi)$, and enclosures of $\cos, \sin$ over a θ -cell give an enclosure of all circle points of that cell. Cells are bisected adaptively: a cell whose subtree is not entirely killed is split until a width floor is reached (surviving cells are reported, never dropped). A runtime canary aborts the whole computation if a circle point ever fails its own defining constraints — a guard against implementation error, not a mathematical step.

B.5 Near-tangent placements

When the quadratic’s discriminant straddles zero (two roots merging), the root enclosure degenerates to a segment $x_p + t n$, $t \in [t]$. Inside sufficiently refined circle cells this segment is subdivided in t like a one-dimensional cell stage, which separates the near-coincident and mirror-image solution regimes that a single fat box cannot distinguish.

B.6 Distinctness: discarding forced coincidences

The candidate graphs admit *degenerate* edge-exact configurations with coincident non-adjacent vertices (fewer than 13 distinct points). Two rules discard branches that cannot contain an *injective* realization; both only ever discard, never affect kill soundness.

Rule 1 (root separation). Let $u \not\sim v$ with overlapping boxes, and let Q be four placed common neighbours. In any true solution both x_u and x_v solve the Q -sphere system, which has at most two solutions — the two roots above. If the two root enclosures are disjoint and the boxes of x_u and x_v meet the *same single* root enclosure, then $x_u = x_v$ in every true solution of the branch, which therefore contains no 13-distinct-point realization. (If either box meets neither root, the branch is empty outright.)

Rule 2 (bisector hyperplane). If $x_u \neq x_v$ and w is a common neighbour, then $\|x_w - x_u\| = \|x_w - x_v\| = 1$ places x_w on the perpendicular bisector hyperplane of $x_u x_v$. Hence if five placed common neighbours have certifiably nonzero affine volume (an interval 4×4 determinant), no hyperplane contains them all and $x_u = x_v$ is forced.

Both rules are applied at placement time and again as a filter at complete leaves. Together with the branch elimination they yield: *if every branch of the search dies or is discarded, the graph admits no realization with 13 distinct points.* A cell that can neither be killed nor discarded at the width floor is reported as a survivor; the final computation has none.

C Search orchestration: slicing and decompositions

(Implemented in `reproduce.py`; C driver `cdriver.py`.)

C.1 Elimination orders

An *elimination order* fixes the seed clique and the sequence of vertex placements; each next vertex has ≥ 4 (sphere step) or exactly 3 (circle step) already-placed neighbours. Orders are generated greedily, *circle-late*: the placed set is closed under sphere steps before a circle vertex is chosen (several tie-breaking heuristics give a small pool of distinct decompositions per graph). The order affects only the shape and cost of the search tree, never the soundness of a verdict; hence *any* single order whose search is fully killed certifies the graph.

C.2 Slicing the circle parameter

The engine accepts a restriction of the *first* circle stage to a subinterval $[a, b] \subseteq [0, 2\pi]$. If a family of closed slices covers $[0, 2\pi]$ and each sliced search is fully killed, the graph is certified: every hypothetical realization has some $\theta \in [0, 2\pi]$ and falls in some slice. Slices are independent, so they parallelize perfectly across cores.

C.3 Recursive refinement

Each slice is run with a node budget. A slice that exceeds its budget (or reports survivors) is split into 8 subslices and retried. Empirically this is the dominant accelerator: with finer initial slices the engine’s very first cells are already narrow enough for immediate certified kills, avoiding the repeated re-exploration that adaptive splitting of coarse cells entails — pieces that ran for hours at coarse slicing die in milliseconds two refinement levels down. Fifty-six graphs need no slicing at all; two need one or two levels; the graph with two circle stages (a two-parameter search) needed three levels on part of its range. The driver therefore starts two-parameter searches directly at fine initial slicing.

C.4 Alternate decompositions

One graph exhibits, under its default decomposition, an *identically-degenerate tangency channel*: along a narrow θ -arc the discriminant of the relevant sphere systems vanishes identically, two vertex pairs coincide exactly (a degenerate configuration family), and the coincidence rules of §B.6 are structurally inconclusive there (merged roots; vanishing affine volumes). Rather than

analyse the channel, the driver races all candidate decompositions concurrently: a different elimination order re-parametrizes the geometry, the degenerate family no longer sits at a tangency of the new parametrization, and the standard machinery certifies all 24 slices of the full circle directly. The final certificate for that graph rests entirely on the alternate decomposition and is self-contained.

C.5 Reproduction

The package provides: `enumerate_aeq.py` (Appendix A; also verifies all published counts), `ckernel.c` (the certified engine, Appendix B) with `cdriver.py/ival.py` (compilation, seed enclosures, elimination orders), the graph list `aeq_d4_n13.json`, and `reproduce.py`, which runs the two validation controls (Lemma-12 graph killed; cross-polytope graph preserved) and then certifies all 59 graphs by the strategy above. The kernel compiles automatically (any C compiler on POSIX; MSVC, clang or gcc on Windows); there are no further dependencies beyond Python ≥ 3.9 . A full run takes on the order of ten minutes on a modern 16-core machine (the original campaign, which predated the driver’s fine-slicing default and concurrent-race cancellation, took on the order of half a day for the hardest graphs on 12 cores); the computation is deterministic, and was observed to reproduce bit-identical search trees across the three compiler/libm/ISA stacks tested.